

SQL

1. What is a database?

There are many definitions for a database. We will use a simple definitions

“A database is an organized collection of data, generally stored and accessed electronically from a computer system.”

There are many examples of a database around us, admission application's database, fee collection database and so on.

But why do we need the database?

2. Need for database

A database is typically used to make the storage and the access of information quicker and efficient. Over and above this there are many advantages of having computerised data. Some of them are

- (a) **Controlled redundancy** : Data redundancy is defined as duplication of data. Some data keeps getting repeated in many places. This results in increase in storage space requirement. Also it may result in data inconsistency. A database by its design controls and limits the data duplication to only where and what is necessary.
- (b) **Data consistency** : Data consistency can be defined as , ‘Same data drawn from separate places must not conflict with each other.’ This does not relate to correctness of data but the same data being available at all access points and all times.
- (c) **Data integrity** : Data integrity refers to data being accurate and consistent in the database. It is very important that the data integrity is maintained through various rules.
- (d) **Data Security** : There are many ways that data stored in database can be secured. There can be multiple users with different kinds of access. Only authorised users can access the database.
- (e) **Standard structure** : Data stored in database must follow a certain standard structure. This makes the data more understandable and portable.

3. RDBMS (Relational Database Management Software)

A Relational database management system (RDBMS) is a database management system (DBMS) that is based on the relational model as introduced by E. F. Codd. A relational database has following major components:

3.1 Table

A relational model has a table as a basic structure to store data. A table is arrangement of rows and column to store data. Table is identified by its name. Here is an example of RDBMS table;

pid	pname	oname	species	gender	dob
1	Fluffy	Kumar	Cat	F	2010-02-04
2	Claws	Radha	Cat	M	2014-03-17
3	Rocky	Vignesh	Dog	M	2014-05-13
4	Berry	Jayanthi	Dog	F	2017-08-31
5	Pluto	Madhu	Dog	M	2016-03-30

3.2 Record or Tuple

Each row in the table is known as a tuple. Following image shows a row in the table. This is also called a record.

pid	pname	oname	species	gender	dob
1	Fluffy	Kumar	Cat	F	2010-02-04
2	Claws	Radha	Cat	M	2014-03-17
3	Rocky	Vignesh	Dog	M	2014-05-13
4	Berry	Jayanthi	Dog	F	2017-08-31
5	Pluto	Madhu	Dog	M	2016-03-30

The number of rows of a table is cardinality of the table. The above table has cardinality 5.

3.3 Field or Column name or Attribute

Each column in the table is called attribute. The attributes are the description of the entity. Here is the example a pet is described by the identification number, name of the pet, owner's name, species of the pet, the gender of the pet and pet's date of birth.

pid	pname	oname	species	gender	dob
1	Fluffy	Kumar	Cat	F	2010-02-04
2	Claws	Radha	Cat	M	2014-03-17
3	Rocky	Vignesh	Dog	M	2014-05-13
4	Berry	Jayanthi	Dog	F	2017-08-31
5	Pluto	Madhu	Dog	M	2016-03-30

The number of columns of the table is it's degree. The degree of the above table is 6.

3.4 Domain

A domain is a set of permitted values for an attribute in table. For example, a domain of month-of-year can accept January, February,...December as values. An attribute cannot accept values that are outside of their domains.

3.5 Keys

Key plays an important role in relational database; it is used for identifying unique rows from table. It also establishes relationship among tables.

We are going to discuss four keys.

Primary Key : A primary is a column or set of columns in a table that uniquely identifies tuples (rows) in that table.

pid	pname	oname	species	gender	dob
1	Fluffy	Kumar	Cat	F	2010-02-04
2	Claws	Radha	Cat	M	2014-03-17
3	Rocky	Vignesh	Dog	M	2014-05-13
4	Berry	Jayanthi	Dog	F	2017-08-31
5	Pluto	Madhu	Dog	M	2016-03-30

Observe that the values of this column are unique and not empty.

Candidate key : These are the column or set of columns that qualify to be primary key. The qualification for being a primary key is unique and non-null values in the column.

Alternate keys : From all the candidate keys one key is selected for being primary key all the remaining candidate keys are called alternate key.

Foreign key : Foreign keys are the columns of a table that points to the primary key of another table. Values of foreign key are derived from primary key of another table. They set relationship between tables.

Table name : Medical

mid	dateofrec	height	weight	pid
11	2020-01-14	67.4	13.5	1
21	2020-01-18	55.4	15.5	2

Table name : Pet

pid	pname	oname	species	gender	dob
1	Fluffy	Kumar	Cat	F	2010-02-04
2	Claws	Radha	Cat	M	2014-03-17
3	Rocky	Vignesh	Dog	M	2014-05-13
4	Berry	Jayanthi	Dog	F	2017-08-31
5	Pluto	Madhu	Dog	M	2016-03-30

They also enforce certain conditions. In the above table the primary table is Pet and Medical is a dependent table. The values of pid in Medical are derived from value of pid in Pet table.

SQL Introduction

1. Structured Query Language or SQL is a standard Database language which is used to create, maintain and retrieve the data from relational databases. There are many software which use SQL as language of coding. Few of them are

- MySQL
- Oracle
- SQL Server

Some of the features of this language are,

- SQL is not case sensitive language. But as convention the keywords are written using uppercase letters while rest code is written in lowercase.
- SQL is interpreter based language.
- It has wide acceptability. SQL is compatible with databases like MS Access, Microsoft SQL Server, MySQL, Oracle.
- SQL has very high performance. It can take very high workload and big databases.
- SQL is an open-source programming language for building relational database management system

2. Categories of SQL commands

The commands of SQL are called as queries. The queries are categorised as follows

- DDL (Data Definition Language)

These commands are used to define the structure of the database like database, tables etc. Some of the DDL commands are

CREATE TABLE, CREATE DATABASE, DROP TABLE etc

- DML

These commands are used to manipulate data in the relation. Some of the DML commands are

INSERT, DELETE, DELETE

- TCL

TCL commands can only use with DML commands like INSERT, DELETE and UPDATE only. Some TCL commands are

COMMIT, ROLLBACK

- DCL

DCL commands are used to grant and take back authority from any database user. Some DCL commands are

GRANT, REVOKE

- DQL
These commands are used to extract the data from the relations. There is only one DQL command and that is

SELECT

3. Installing MySQL

As per CBSE syllabus we are required to use MySQL. We are going to see how to install this software both in Windows and Linux OS.

3.1 Windows installation

To download MySQL installer, go to the following link

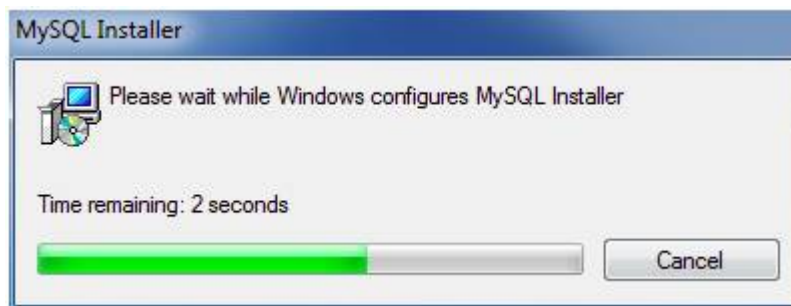
<http://dev.mysql.com/downloads/installer/>.

There are two installer files, you can download the following file.

mysql-installer-community-<version>.exe

Choose appropriate version according to your OS version.

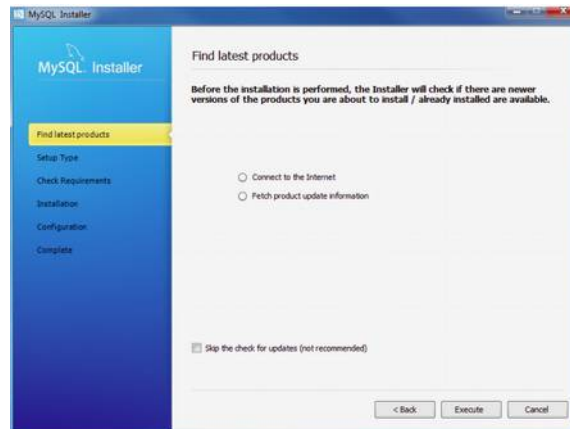
To install MySQL using the MySQL installer, double-click on the MySQL installer file and follow the steps below:



Step 1: Choose the first option Install MySQL products

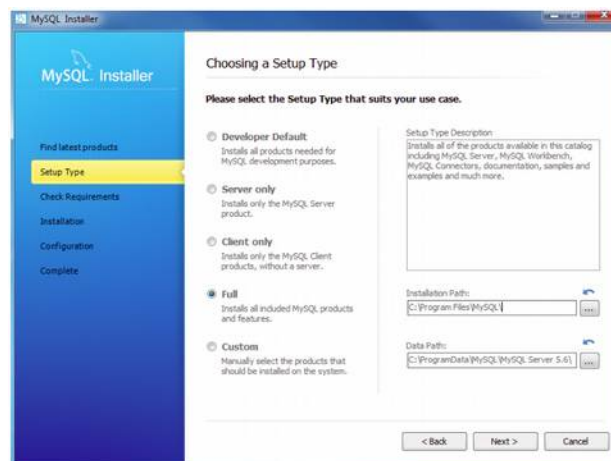


Step 2 : MySQL installer checks and downloads the latest MySQL products including MySQL server, MySQL Workbench, etc.

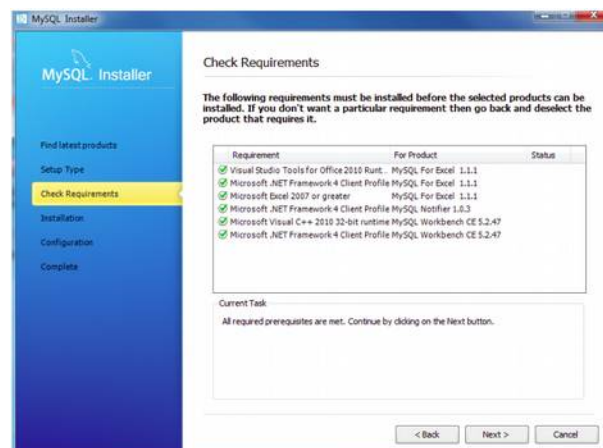


Step 3 : Click on next in the above window after product update is done.

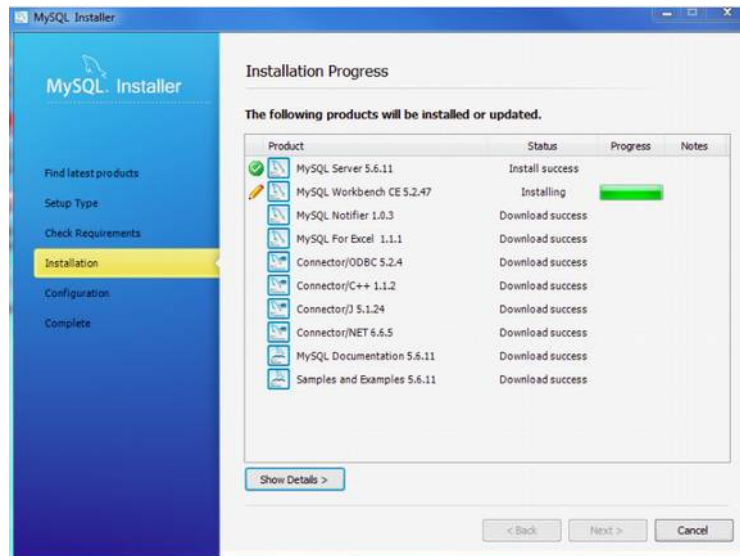
Step 4 : Choosing a Setup Type: there are several setup types available. Choose the Full option to install all MySQL products and features.



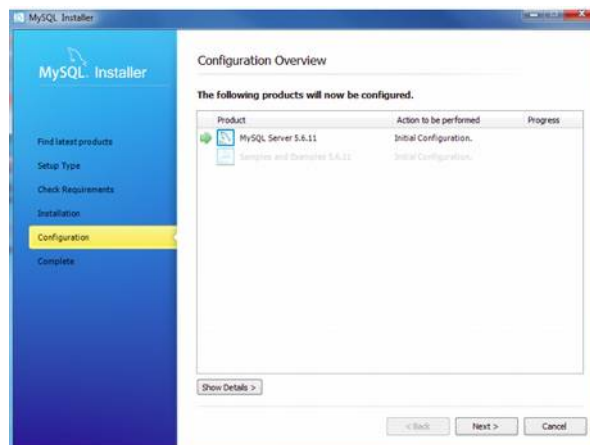
Step 5 : MySQL installer will now check for requirements.



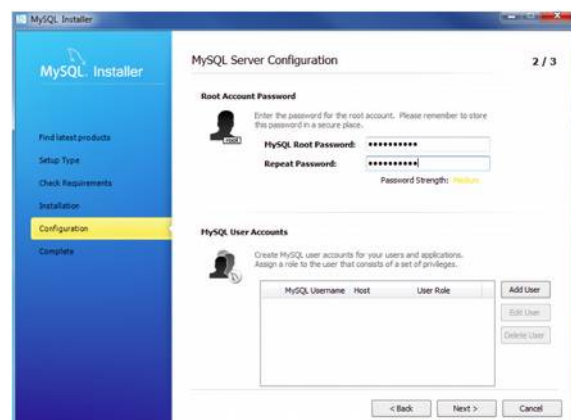
Step 6 : MySQL Installer downloads all selected products. It will take a while, depending on the speed of your internet connection.



Step 7 : When all the installations are successfully completed click next. Now the server would be configured.



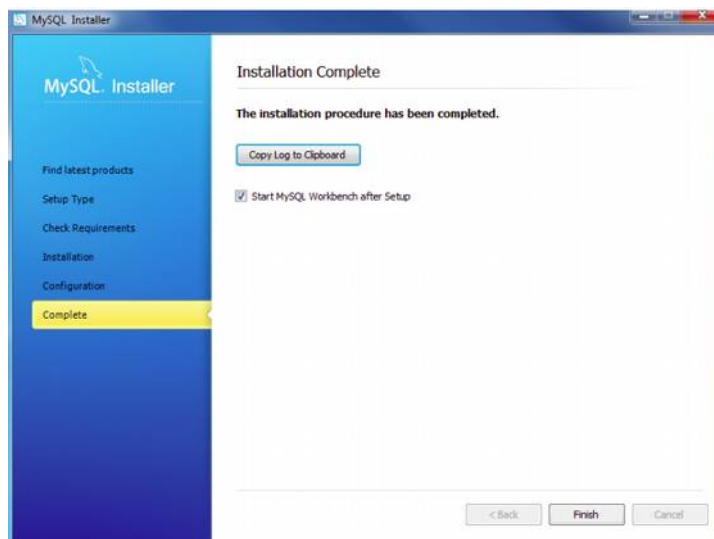
Step 8 : You would be prompted for username and password for root user.



Step 9 : Complete the server configuration and click next. Remember to click the checkbox 'Start the MySQL Server at System Startup'.



Step 10 : After completing the configuration of server and Samples and Examples click next. You have completed the installation successfully. Click on Finish.



3.2 Linux Installation

Step 1 : Update the package with commands

```
sudo apt-get update
```

Step 2: Upgrade the packages with commands

```
sudo apt-get upgrade
```

Step 3: Using the following command install MySQL

```
sudo apt-get install mysql-server
```

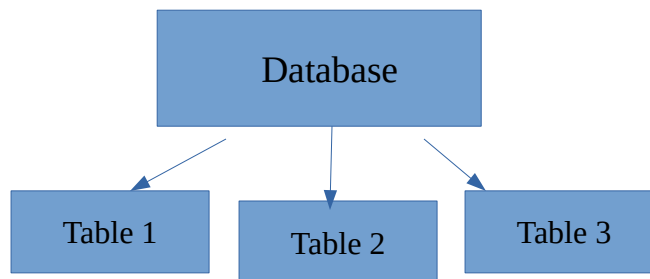

Step 4: Start working with MySQL using following command.

```
mysql -u root -p
```

4. Working with Database

After the installation of MySQL has been completed successfully we can start working with SQL commands.

In relational database model a database is a structure that contains table. Every database can have many tables. So we have to create a database first.



The command to create a database is

```
CREATE DATABASE databasename;
```

Every SQL command ends with a semi-colon.

Example

```
mysql> CREATE DATABASE classroom;  
Query OK, 1 row affected (0.00 sec)
```

The list of existing databases can be seen using command

```
SHOW DATABASES;
```

```
mysql> show databases;  
+-----+  
| Database           |  
+-----+  
| information_schema |  
| classroom          |  
| contactdb          |  
| mysql              |  
| performance_schema |  
| school             |  
| spellgame          |  
| student            |  
| sys                |  
| testdb             |  
+-----+
```

We may need to remove the database already existing in the list and the command for that is;

```
DROP DATABASE databasename;
```

Examples

```
mysql> DROP DATABASE student;  
Query OK, 3 rows affected (1.15 sec)
```

3 rows affected indicates that there were three tables existing in the database which got dropped.

When the database is ready, there is need to make it active. This can be done using commands

```
USE databasename;
```

Example

```
mysql> use classroom;  
Reading table information for completion of table and column names  
You can turn off this feature to get a quicker startup with -A
```

Database changed

Recap of all the commands that we learnt

1. CREATE DATABASE databasename;
2. SHOW DATABASES;
3. DROP DATABASE databasename;
4. USE databasename;

5. Working with Tables

- 5.1 The first step to work with tables is to create one. Creating a basic table involves naming the table and defining its columns and each column's data type. The syntax of command to create a table is;

```
CREATE TABLE table_name(  
column1 datatype constraint,  
column2 datatype constraint,  
column3 datatype constraint,  
.....  
columnN datatype constraint,
```

PRIMARY KEY(one or more columns));
CREATE TABLE is the command telling the database system to create a new table.
The name of the table must be unique in a given database.

5.1.1 Datatypes in MySQL

Datatype of the column is decided based on the domain of the column. Some of the commonly used datatypes are

Datatype	From	To
int	-2,147,483,648	2,147,483,647
decimal(p,d)	$-10^{38} + 1$	$10^{38} - 1$
float	$-1.79E + 308$	$1.79E + 308$
date	Stores dates in "yyyy-mm-dd" format	
char	Maximum length of 255 characters.(Fixed length)	
varchar	Maximum of 255 characters.(Variable-length)	

There can be minor variation in the ranges based on the version of MySQL being used and system you are working on.

Let us now discuss these datatypes in detail.

INT : This datatype stores integers i.e. a number without fraction. Integer can be zero, positive or negative. In CREATE TABLE command this datatype can be specified as 'INT' or as 'INTEGER'

The maximum digits for INT can be specified inside parenthesis. By default this value is 11 that means 11 digits can be displayed.

DECIMAL (P,D) : This datatype is used to store fractions. The syntax for this is

column_name decimal(P,D)

In the syntax above:

- P is the precision that represents the number of significant digits. The range of P is 1 to 65.
- D is the scale that represents the number of digits after the decimal point. The range of D is 0 and 30.

D should be less than or equal to P. The actual range of the decimal column depends on the precision and scale.

FLOAT(n) : Float is a single precision floating point number.

DATE : MySQL uses yyyy-mm-dd format for storing a date value. This format is fixed and it is not possible to change it, but the display format can be changed.

CHAR (n) : The CHAR data type is a fixed-length character type in MySQL. You often declare the CHAR type with a length that specifies the maximum number of characters that you want to store. For example, CHAR(20) can hold up to 20 characters. The storage requirements for a CHAR is fixed irrespective of the specific string stored.

If the data that you want to store is a fixed size, then you should use the CHAR data type.

VARCHAR(n) : MySQL VARCHAR is the variable-length string up to 255 characters. VARCHAR is declared with a length that specifies the maximum number of characters that you want to store. For example, CHAR(20) can hold up to 20 characters. The storage requirements for a VARCHAR vary depending on the specific string stored.

If the data that you want to store is a fixed size, then you should use the CHAR data type.

5.1.2 Constraints in MySQL

The constraints in CREATE TABLE command puts some restrictions on the column. This is important for maintaining data integrity. Let us discuss some constraints in detail

PRIMARY KEY : A PRIMARY KEY constraint for a table enforces the table to accept unique and not null data for a specific column.

NOT NULL : This constraint allows to specify that a column can not contain any NULL value. A null value means absence of a value.

UNIQUE : The UNIQUE constraint does not allow to insert a duplicate value in a column. The difference between UNIQUE and PRIMARY KEY is that UNIQUE column can have null values while PRIMARY KEY cannot have null values.

DEFAULT : While inserting data into a table, if no value is supplied to a column, then the column gets the value set as DEFAULT.

For example

```
country varchar(25) NOT NULL DEFAULT 'India',
```

Here if the user does not give input for country column then it will take value 'India'.

Now let us create tables with these datatypes and constraints.

```
CREATE TABLE CUSTOMERS(
  ID INT          PRIMARY KEY,
  NAME VARCHAR (20) NOT NULL,
  AGE INT         NOT NULL,
  ADDRESS VARCHAR (25) ,
  SALARY DECIMAL (9, 2)
);
```

Observe the way the primary key is defined.

The same command can also be written as

```
CREATE TABLE CUSTOMERS(
  ID INT          ,
  NAME VARCHAR (20) NOT NULL,
  AGE INT         NOT NULL,
  ADDRESS VARCHAR (25) ,
  SALARY DECIMAL (9, 2),
  PRIMARY KEY (ID)
);
```

Observe the way the primary key is defined. This way can be adopted when more than one column make a primary key.

5.1.3 View the structure of the table

After the table is created we need to verify the structure that is created. The following command can be used for the this.

```
DESCRIBE table_name;
```

Examples

```
mysql> DESCRIBE CUSTOMERS;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| ID    | int(11)       | NO   | PRI | NULL    |       |
| NAME  | varchar(20)   | NO   |     | NULL    |       |
| AGE   | int(11)       | NO   |     | NULL    |       |
| ADDRESS | varchar(25)   | YES  |     | NULL    |       |
| SALARY | decimal(9,2)  | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

5.2 Delete a table

If the table created is no more required then it can be completely deleted using command;

```
DROP TABLE table_name;
```

Example

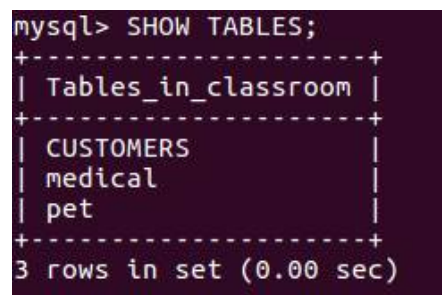
```
mysql> DROP TABLE CUSTOMERS;  
Query OK, 0 rows affected (0.17 sec)
```

5.3 View the tables in the databases

To view the list of tables existing in the active database following command can be used.

```
SHOW TABLES;
```

Example



```
mysql> SHOW TABLES;  
+-----+  
| Tables_in_classroom |  
+-----+  
| CUSTOMERS           |  
| medical              |  
| pet                  |  
+-----+  
3 rows in set (0.00 sec)
```

5.4 Alter the structure of a table

Sometimes there is need to change the structure of the table. Some of the reasons for this are;

- Change in organizational rules. E.g. admission number that was 5 digits may need to be 6 digits now.
- Error in create table command.

Alter table command can do following

5.4.1 Add a column

The syntax to add a column is

```
ALTER TABLE table_name  
ADD  
new_column_name datatype constraint;
```

5.4.2 Modify a column

The syntax to modify a column is

```
ALTER TABLE table_name  
MODIFY  
new_column_name datatype constraint;
```

5.4.3 Delete a column in a table

The syntax

```
ALTER TABLE table_name  
DROP column_name;
```

Recap of the commands learnt

1. CREATE TABLE
2. DESCRIBE
3. DROP TABLE
4. SHOW TABLES;
5. ALTER TABLE

Data Manipulation – Part 1

1. Populate the table

In the previous session we have seen how to create a table. This is an empty structure that need to be populated with data.

This task is done using INSERT command. The syntax of this command is

```
INSERT INTO table_name (column1, column2, column3, ...)  
VALUES (value1, value2, value3, ...);
```

If you are adding values for all the columns of the table, you do not need to specify the column names in the SQL query. However, make sure the order of the values is in the same order as the columns in the table.

The syntax for this variation is

```
INSERT INTO table_name  
VALUES (value1, value2, value3, ...);
```

Observe the change that in the second syntax there is no list of columns.

Example

To see the example of this command we will see one table that we have already created. We can check the structure of the table before giving the insert command in case you are not sure about the names of columns, data types and constraints.

mysql> describe pet;

Field	Type	Null	Key	Default	Extra
pid	float	NO	PRI	NULL	
pname	varchar(20)	YES		NULL	
oname	varchar(30)	YES		NULL	
species	varchar(20)	YES		NULL	
gender	char(1)	YES		NULL	
dob	date	YES		NULL	

Now that the structure is available with us, we can give the command.

```
INSERT INTO pet (pid,pname,oname,species,gender,dob)
VALUES (1,"Jiffy","Raju","Dog","M","2000/12/03");
```

pid	pname	oname	species	gender	dob
1	Jiffy	Raju	Dog	M	2000-12-03

1 row in set (0.00 sec)

For each row to be inserted separate insert command has to be given.

```
INSERT INTO pet
VALUES (2,"Tuffy","Rajesh","Dog","M","2002/10/03");
```

After this second insert the table will be

pid	pname	oname	species	gender	dob
1	Jiffy	Raju	Dog	M	2000-12-03
2	Tuffy	Rajesh	Dog	M	2002-10-03

2 rows in set (0.00 sec)

Now let us see how to add NULL values in the table. A field with a NULL value is a field with no value.

```
INSERT INTO pet (pid,pname,oname,species,gender)
VALUES (3,"Kitty","Parag","Cat","F");
```

We have not given the value of date of birth, therefore the value inserted in the table is NULL.

pid	pname	oname	species	gender	dob
1	Jiffy	Raju	Dog	M	2000-12-03
2	Tuffy	Rajesh	Dog	M	2002-10-03
3	Kitty	Parag	Cat	F	NULL

3 rows in set (0.00 sec)

So if any column's value is not available and the column accepts NULL value the NULL value can be inserted by skipping the column from the list of column. If the column list is not given then the values of all the columns must be given.

All the values other than numerical values must be enclosed in either single quotes or double quotes. The dates are also to be enclosed in either single quotes or double quotes.

2. Retrieve data from table

The command used to retrieve required data is SELECT. The basic syntax of the statement is;

```
SELECT column1, column2, ...  
FROM table_name;
```

The list of columns given here are the columns chosen to display. From clause specifies the table from where the data is to be retrieved.

Now let us take the 'pet' table as example and learn more about SELECT command.

The table has following data

pid	pname	oname	species	gender	dob	weight
1	Jiffy	Raju	Dog	M	2000-12-03	45.2
2	Tuffy	Rajesh	Dog	M	2002-10-03	23.5
3	King	Mohan	Dog	M	2011-03-17	35.4
4	Teddy	Kiran	Cat	M	2012-01-02	12.2
5	Fido	Raman	Dog	F	2010-07-12	37.4
6	Wooly	Suman	Cat	F	2011-05-05	10.9
7	Buster	Aman	Dog	M	2009-05-15	28.1

7 rows in set (0.00 sec)

2.1 Select columns from the table

The column list can be given to select columns from the table.

Example

```
mysql> select pid,pname from pet;
+-----+-----+
| pid | pname |
+-----+-----+
| 1   | Jiffy |
| 2   | Tuffy |
| 3   | King  |
| 4   | Teddy |
| 5   | Fido  |
| 6   | Wooly |
| 7   | Buster|
+-----+-----+
7 rows in set (0.00 sec)
```

Only two columns were selected.

If we want to select all the all columns of the table then there is no need to write complete list. This task can be done using '*'. When we replace the column list with '*' all the columns of the table are displayed.

```
mysql> select * from pet;
+-----+-----+-----+-----+-----+-----+-----+
| pid | pname | oname | species | gender | dob       | weight |
+-----+-----+-----+-----+-----+-----+-----+
| 1   | Jiffy | Raju  | Dog     | M      | 2000-12-03 | 45.2   |
| 2   | Tuffy | Rajesh| Dog     | M      | 2002-10-03 | 23.5   |
| 3   | King  | Mohan | Dog     | M      | 2011-03-17 | 35.4   |
| 4   | Teddy | Kiran | Cat     | M      | 2012-01-02 | 12.2   |
| 5   | Fido  | Raman | Dog     | F      | 2010-07-12 | 37.4   |
| 6   | Wooly | Suman | Cat     | F      | 2011-05-05 | 10.9   |
| 7   | Buster| Aman  | Dog     | M      | 2009-05-15 | 28.1   |
+-----+-----+-----+-----+-----+-----+-----+
7 rows in set (0.00 sec)
```

Using the column list we can also perform some basic calculations. The operators permitted here are

Operator	Description
+	Add
-	Subtract
*	Multiply
/	Divide

Example

We are going to display 10% of the weight of the pet to decide the dosage of some medicine. How will you calculate 10% of weight?

```
mysql> select pname,weight*0.1 from pet;
```

pname	weight*0.1
Jiffy	4.52
Tuffy	2.35
King	3.54
Teddy	1.22
Fido	3.74
Wooly	1.09
Buster	2.81

7 rows in set (0.00 sec)

Observe the calculation and heading of the column given.

If you would have observed the heading of the column is exactly as it is written in the command. But we may like to change it to a meaningful name. This can be done using column alias.

```
mysql> select pname,weight*0.1 as Dosage from pet;
```

pname	Dosage
Jiffy	4.52
Tuffy	2.35
King	3.54
Teddy	1.22
Fido	3.74
Wooly	1.09
Buster	2.81

7 rows in set (0.00 sec)

Observe the alias column name given.

Column alias is

- Renaming of column heading
- useful while doing calculations
- Immediately follows the column name with 'AS' keyword.
- Requires inverted quotes if it contains spaces or special characters.

If there are any NULL values in the column on which the calculations are performed

- Arithmetic expression containing a NULL value will be evaluated to NULL

DISTINCT keyword can be used with column to display unique values of a column.

Example

```
mysql> select DISTINCT species from pet;
```

species
Dog
Cat

2 rows in set (0.01 sec)

2.1.1 Where clause

In all the previous examples you would have observed that all the rows of the table are getting displayed. But most of the times we need to display only those rows which satisfy certain condition.

To perform this task we are going to add one clause to the existing command and that clause is 'WHERE'. The syntax for this is

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition(s);
```

Conditions can be specified using

Operator	Meaning
=	Equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to
<>	Not equal to
BETWEEN ...AND...	Between two values (inclusive)
IN(set)	Match any of a list of values
LIKE	Match a character pattern
IS NULL	Is a null value

The logical operators can be used to combine the above conditions

Operator	Meaning
AND	Returns TRUE if <i>both</i> component conditions are true
OR	Returns TRUE if <i>either</i> component condition is true
NOT	Returns TRUE if the following condition is false

While working with multiple operators the precedence order followed is as follows

Operator	Meaning
1	Arithmetic operators
2	Concatenation operator
3	Comparison conditions
4	IS [NOT] NULL, LIKE, [NOT] IN
5	[NOT] BETWEEN
6	Not equal to
7	NOT logical condition
8	AND logical condition
9	OR logical condition

We will look at few examples now.

Example.

- a. Display pet names of dogs.

```
SELECT pname
FROM pet
WHERE species="Dog";
```

You should write queries with more than one clause on multiple lines to make it more readable.

```
+-----+
| pname |
+-----+
| Jiffy |
| Tuffy |
| King  |
| Fido  |
| Buster|
+-----+
5 rows in set (0.00 sec)
```

Only the rows that satisfy the conditions are displayed.

- b. Display pet id and pet name of pets whose weight is less than 20kg.

```
SELECT pid, pname
FROM pet
WHERE weight < 20;
```

```
+-----+-----+
| pid | pname |
+-----+-----+
| 4   | Teddy |
| 6   | Wooly |
+-----+-----+
2 rows in set (0.00 sec)
```


- c. Display the owner's name of female pets who are more than 15kg.

```
SELECT oname
FROM pet
WHERE gender='F' and weight >15;
```

```
+-----+
| oname |
+-----+
| Raman |
+-----+
1 row in set (0.00 sec)
```

- d. Display the details of pets whose weight is in the range of 15 to 25 (both inclusive)

```
SELECT *
FROM pet
WHERE weight >=15 and weight <=25;
```

```
+-----+-----+-----+-----+-----+-----+-----+
| pid | pname | oname | species | gender | dob       | weight |
+-----+-----+-----+-----+-----+-----+-----+
| 2   | Tuffy | Rajesh | Dog      | M      | 2002-10-03 | 23.5   |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Here we will demonstrate the usage of BETWEEN keyword.

When the condition is a range with both the upper and lower boundary inclusive BETWEEN keyword can be used. BETWEEN can be used with numbers and dates.

```
SELECT *
FROM pet
WHERE weight BETWEEN 15 and 25;
```

This command will give exactly same output as above.

```
+-----+-----+-----+-----+-----+-----+-----+
| pid | pname | oname | species | gender | dob       | weight |
+-----+-----+-----+-----+-----+-----+-----+
| 2   | Tuffy | Rajesh | Dog      | M      | 2002-10-03 | 23.5   |
+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

IN keyword is used to specify membership of a collection. See the command given below

```
SELECT *  
FROM pet  
WHERE species IN("Fish","Bird");
```

```
mysql> select * from pet where species IN ("Fish","Bird");  
Empty set (0.00 sec)
```

As there are no fish or bird as pets in the table presently the result is "Empty Set".

LIKE keyword is used to do pattern matching. Two symbols are used for pattern matching

% - Replaces any number of characters
_ - Replace one character.

So to display the names of owners of pets that start with R the pattern will be "R%" i.e. R followed by any number of characters. The command can be written as

```
SELECT oname  
FROM pet  
WHERE oname LIKE "R%";
```

```
+-----+  
| oname |  
+-----+  
| Raju  |  
| Rajesh|  
| Raman |  
+-----+  
3 rows in set (0.00 sec)
```

Only names starting with
'R' are selected.

IS NULL is a special check for NULL values. As NULL value is not comparable there is a special way to check if any value is missing. (The table is slightly modified with a row with NULL value to demonstrate this)

There is a pet whose date of birth is not known and therefore it is NULL also the weight is unknown.

So to display the list of pets whose date of birth is unknown the command would be

```
SELECT *  
FROM pet  
WHERE dob IS NULL;
```

```
+-----+-----+-----+-----+-----+-----+-----+  
| pid | pname | oname | species | gender | dob | weight |  
+-----+-----+-----+-----+-----+-----+-----+  
| 8 | Marsha | Sheetal | Cat | F | NULL | NULL |  
+-----+-----+-----+-----+-----+-----+-----+  
1 row in set (0.00 sec)
```


2.1.2 Ordering the result

We have seen that the result of query that is given is not in any order. To sort the retrieved rows in either ascending or descending order ORDER BY clause is used. The syntax of the command is

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition(s)  
ORDER BY column1, column2, ... ASC|DESC;
```

These are optional clauses. They are to be used only when required.

Example

- a. Display the names, date of birth and weights of pets in ascending order of their weights.

```
SELECT pname, dob, weight  
FROM pet  
ORDER BY weight ASC;
```

pname	dob	weight
Marsha	NULL	NULL
Wooly	2011-05-05	10.9
Teddy	2012-01-02	12.2
Tuffy	2002-10-03	23.5
Buster	2009-05-15	28.1
King	2011-03-17	35.4
Fido	2010-07-12	37.4
Jiffy	2000-12-03	45.2

8 rows in set (0.00 sec)

Observe the ascending order of weight and position of NULL.

- b. Display the names, date of birth and weights of pets in descending order of their weights.

pname	dob	weight
Jiffy	2000-12-03	45.2
Fido	2010-07-12	37.4
King	2011-03-17	35.4
Buster	2009-05-15	28.1
Tuffy	2002-10-03	23.5
Teddy	2012-01-02	12.2
Wooly	2011-05-05	10.9
Marsha	NULL	NULL

8 rows in set (0.00 sec)

weight

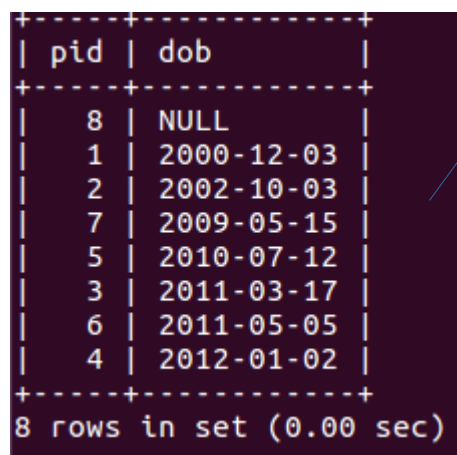
```
SELECT pname, dob,  
FROM pet  
ORDER BY weight DESC;
```

Observe the descending order of weight and position of NULL.

Note that NULL value is taken as the smallest value. Also we will work with dates in ORDER BY clause.

- c. Display the pid and date of birth in ascending order of date of birth.

```
SELECT pid, dob  
FROM pet  
ORDER BY dob ASC;
```



pid	dob
8	NULL
1	2000-12-03
2	2002-10-03
7	2009-05-15
5	2010-07-12
3	2011-03-17
6	2011-05-05
4	2012-01-02

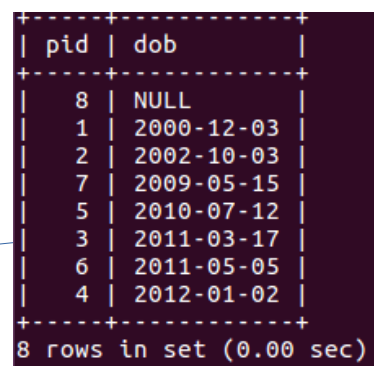
8 rows in set (0.00 sec)

Observe that the oldest date is the smallest date and the latest date is the largest. NULL is smallest.

The default order in ORDER BY is ascending order. That means if no order is specified then the rows would be arranged in ascending order. The above command re-executed without specifying order.

```
SELECT pid, dob  
FROM pet  
ORDER BY dob ;
```

Observe that this is ascending order.



pid	dob
8	NULL
1	2000-12-03
2	2002-10-03
7	2009-05-15
5	2010-07-12
3	2011-03-17
6	2011-05-05
4	2012-01-02

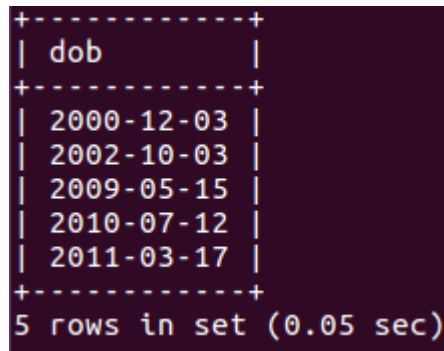
8 rows in set (0.00 sec)

The ORDER BY clause can be combined with WHERE clause. We will see few examples.

Example

- a. Display the DOB of all dogs in ascending order.

```
SELECT dob
FROM pet
WHERE species = "Dog"
ORDER BY dob;
```



The screenshot shows a terminal window with a SQL query result. The query is: SELECT dob FROM pet WHERE species = "Dog" ORDER BY dob;. The result is a table with one column 'dob' and five rows of dates: 2000-12-03, 2002-10-03, 2009-05-15, 2010-07-12, and 2011-03-17. The table is enclosed in a box with dashed lines. Below the table, it says '5 rows in set (0.05 sec)'.

dob
2000-12-03
2002-10-03
2009-05-15
2010-07-12
2011-03-17

5 rows in set (0.05 sec)

DML COMMANDS – PART 2

1. Creating groups of data

The GROUP BY clause groups a set of rows into a set of summary rows by values of columns or expressions. The GROUP BY clause returns one row for each group.

The GROUP BY clause is an optional clause of the SELECT statement. GROUP BY is often used with aggregate functions that we have seen in the last lesson. The syntax of the command is

```
SELECT function(column1), column2, ...
FROM table_name
WHERE condition(s)
ORDER BY column1, column2 ASC/DESC
GROUP BY column1, column2, ... ;
```

The GROUP BY clause must appear after the FROM and WHERE clauses. Following the GROUP BY keywords is a list of comma-separated columns or expressions that you want to use as criteria to group rows.

To work with GROUP BY clause we are going to use the following table;

Table Name : Placement

sno	gender	ssc_p	ssc_b	hsc_p	hsc_b	hsc_s	degree_p	degree_t	workex	sal
1	M	67.00	Others	91.00	Others	Commerce	58.00	Sci&Tech	N	270000
2	M	79.33	Central	78.33	Others	Science	77.48	Sci&Tech	Y	200000
3	M	65.00	Central	68.00	Central	Arts	64.00	Comm&Mgmt	N	250000
4	M	56.00	Central	52.00	Central	Science	52.00	Sci&Tech	N	NULL
5	M	85.80	Central	73.60	Central	Commerce	73.30	Comm&Mgmt	N	425000
6	M	55.00	Others	49.80	Others	Science	67.25	Sci&Tech	Y	NULL
7	F	46.00	Others	49.20	Others	Commerce	79.00	Comm&Mgmt	N	NULL
8	M	82.00	Central	64.00	Central	Science	66.00	Sci&Tech	Y	252000
9	M	73.00	Central	79.00	Central	Commerce	72.00	Comm&Mgmt	N	231000
10	M	58.00	Central	70.00	Central	Commerce	61.00	Comm&Mgmt	N	NULL

10 rows in set (0.00 sec)

Example

- a. Display the average SSC percentage categorised by work experience.

```
SELECT workex,AVG(ssc_p)
FROM placement
GROUP BY workex;
```

workex	AVG(ssc_p)
N	64.400000
Y	72.110000

2 rows in set (0.00 sec)

- b. Display minimum and maximum salary paid to male.

```
SELECT MIN(sal),MAX(sal)
FROM placement
WHERE gender='M'
GROUP BY gender;
```

workex	AVG(ssc_p)
N	64.400000
Y	72.110000

2 rows in set (0.00 sec)

First the grouping is done based on gender then from that using where condition only male members were selected. Now from these selected rows minimum and maximum values decided.

- c. Count number of candidates for placement from each SSC board.

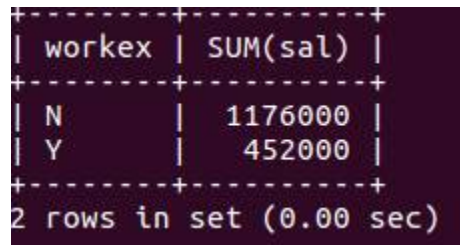
```
SELECT ssc_b,COUNT(*)
FROM placement
GROUP BY ssc_b;
```

ssc_b	COUNT(*)
Central	7
Others	3

2 rows in set (0.00 sec)

- d. Display total salary given to candidates as per their work experience.

```
SELECT workex,SUM(sal)
FROM placement
GROUP BY workex;
```



workex	SUM(sal)
N	1176000
Y	452000

2 rows in set (0.00 sec)

If you carefully observe these examples, you will find that GROUP BY is always used in conjunction with some aggregate function(s). A SELECT command with GROUP BY clause has a column name and one or more aggregate functions which are applied on that column and grouping is also done on this column only.

1.2 Having

Sometimes we do not want to see the whole output produced by a statement with GROUP BY clause. We want to see the output only for those groups which satisfy some condition.

It means we want to put some condition on individual groups (and not on individual records). A condition on groups is applied by HAVING clause. The syntax for this is

```
SELECT function(column1), column2, ...
FROM table_name
WHERE condition(s)
ORDER BY column1, column2 ASC/DESC
GROUP BY column1, column2, ...
HAVING condition(s);
```

This is condition for rows.

This is condition for group.

Example

- a. Display the HSC stream taken by more than one student.

```
SELECT hsc_s
FROM placement
GROUP BY hsc_s
```

HAVING COUNT(*) >1;

```
+-----+
| hsc_s |
+-----+
| Commerce |
| Science |
+-----+
2 rows in set (0.01 sec)
```

The stream 'Arts' that has only one candidate is not displayed as count is not greater than 1.

- b. Display the degree stream whose average salary is greater than 250000

```
SELECT degree_t
FROM placement
GROUP BY degree_t
HAVING AVG(sal)>250000;
```

```
+-----+
| degree_t |
+-----+
| Comm&Mgmt |
+-----+
1 row in set (0.00 sec)
```

Few points to remember about HAVING clause

- HAVING filters records that work on summarized GROUP BY results.
- HAVING applies to summarized group records, whereas WHERE applies to individual records.
- Only the groups that meet the HAVING criteria will be returned.
- HAVING requires that a GROUP BY clause is present.
- WHERE and HAVING can be in the same query.

2. Deleting records

DELETE is a DML command that would delete records from a table. The syntax of the command is

```
DELETE FROM table_name
WHERE condition(s);
```

If no condition is specified then all the records of the table would be deleted.

Some points to remember about DELETE

- DELETE permanently removes records from a table.

- DELETE can delete one or more records in a table.
- Use the WHERE clause to DELETE only specific records.

Example

- a. Delete all the records from placement table.

DELETE FROM placement;

```
mysql> delete from placement;
Query OK, 10 rows affected (0.04 sec)
```

All 10 records
deleted.

- b. Delete candidates who have less than 50% in HSC.

DELETE FROM placement
WHERE hsc_p<50;

```
mysql> delete from placement where hsc_p<50;
Query OK, 2 rows affected (0.28 sec)
```

3. Updating data

When we want to change the existing data in the table we use UPDATE command.
Syntax of this DML command is

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

If you omit the WHERE clause, all records in the table will be updated.

Example

- a. Increase the salary of all those who are having experience by 10000.

```
UPDATE placement
SET sal = sal + 10000
WHERE workex = "Y";
```

```
mysql> UPDATE placement
-> SET sal = sal + 10000
-> WHERE workex = "Y";
Query OK, 2 rows affected (0.84 sec)
Rows matched: 2 Changed: 2 Warnings: 0
```

- b. Increase the percentage of Comm&Mgmt by 5%

```
UPDATE placement  
SET degree_p  
WHERE degree_t="Comm&Mgmt";
```

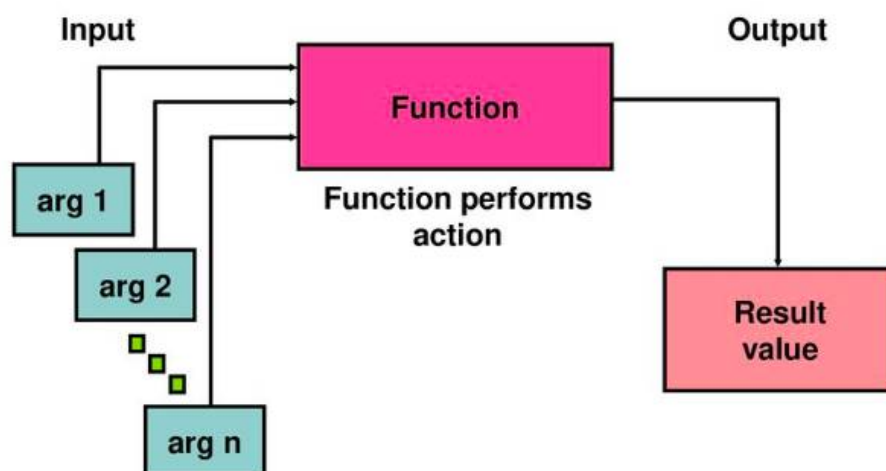
```
mysql> UPDATE placement SET degree_p = degree_p + 0.05* degree_p WHERE degree_t  
="Comm&Mgmt";  
Query OK, 4 rows affected, 1 warning (0.12 sec)  
Rows matched: 4 Changed: 4 Warnings: 1
```

RECAP

1. Group By clause : Grouping records
2. Having : Condition on group of records
3. Delete : Command to delete records permanently
4. Update : Command to change existing data in the table.

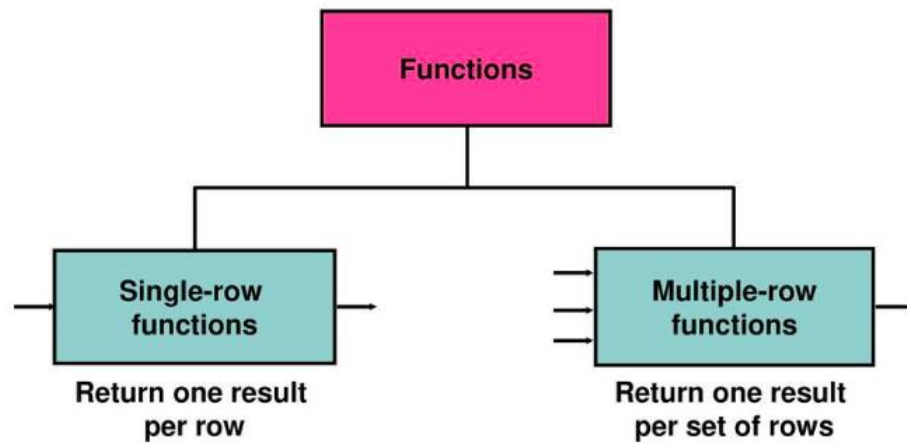
SQL FUNCTIONS

1. SQL has many functions that can be used for statistical or effective extraction of data.



As the diagram shows a SQL function takes number of values performs some operation and gives the result.

The SQL functions are divided in two categories



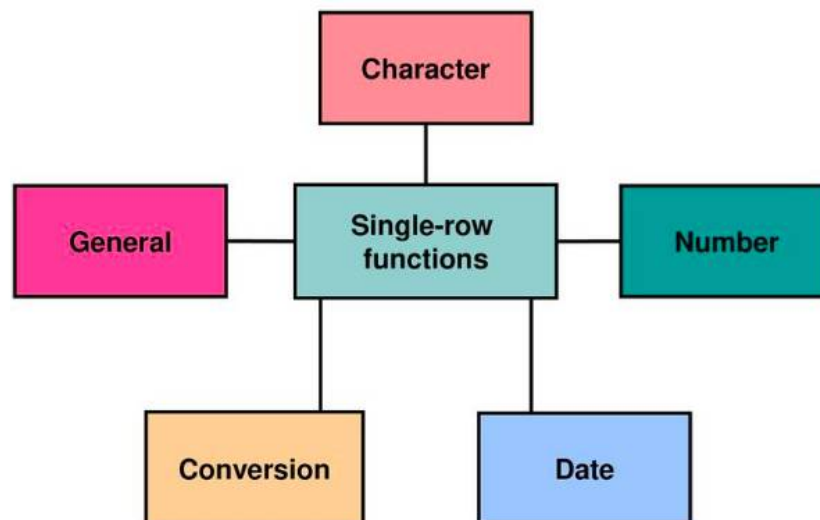
1.1 Single row functions

These function take a row at a time perform the operation and return the value.

Single row functions

- Act on each row.
- Return one result per row

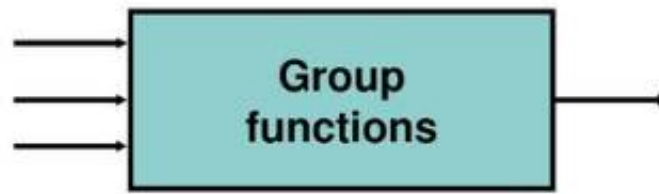
The single row functions are further divided in five categories;



We will be revisiting these functions later in next lesson.

1.2 Multi-row functions

Multi-row or aggregate functions operate on set of rows and return a single result.



The group functions in SQL are

- AVG
- MIN
- MAX
- SUM
- COUNT

The general syntax to use the functions is

```

SELECT function(column1), column2, ...
FROM table_name
WHERE condition(s);

```

We will be looking at them one by one.

To work with these functions we will be using the pet table. Let us see the contents again.

pid	pname	oname	species	gender	dob	weight
1	Jiffy	Raju	Dog	M	2000-12-03	45.2
2	Tuffy	Rajesh	Dog	M	2002-10-03	23.5
3	King	Mohan	Dog	M	2011-03-17	35.4
4	Teddy	Kiran	Cat	M	2012-01-02	12.2
5	Fido	Raman	Dog	F	2010-07-12	37.4
6	Wooly	Suman	Cat	F	2011-05-05	10.9
7	Buster	Aman	Dog	M	2009-05-15	28.1
8	Marsha	Sheetal	Cat	F	NULL	NULL

8 rows in set (0.00 sec)

- **AVG** : This function will give the average of the column values. This can be used with all the numeric values. The NULL values are ignored.

Example

- Display average weight of dogs.

```

SELECT AVG(weight)
FROM pet

```

WHERE species = "Dog";

```

+-----+
| AVG(weight) |
+-----+
|      33.92000 |
+-----+
1 row in set (0.00 sec)

```

- MIN : This function will give the minimum value from the values of column. This function can be used for numeric values and dates. The NULL values are ignored.

Example

- a. Display minimum weight of cats.

```
SELECT MIN(weight)
FROM pet
WHERE species="Cat";
```

```
+-----+
| MIN(weight) |
+-----+
|          10.9 |
+-----+
1 row in set (0.69 sec)
```

- b. Display the date of birth of oldest pet.

```
SELECT MIN(dob)
FROM pet;
```

```
+-----+
| MIN(dob) |
+-----+
| 2000-12-03 |
+-----+
1 row in set (0.00 sec)
```

The oldest date is the minimum and latest date is maximum.

- MAX : This function will give the maximum value from the values of column. This function can be used for numeric values and dates.

Example

- a. Display maximum weight of pets born before 1 Jan 2010.

```
SELECT MAX(weight)
FROM pet
WHERE dob<"2020/01/01";
```

```
+-----+
| MAX(weight) |
+-----+
|          45.2 |
+-----+
1 row in set (0.00 sec)
```

- b. Display the date of birth of youngest pet.

```
SELECT MAX(dob)
FROM pet;
```

```
+-----+
| MAX(dob) |
+-----+
| 2012-01-02 |
+-----+
1 row in set (0.00 sec)
```

- SUM : This function will give the sum of the values of column. This function can be used for numeric values only.

Example

- a. Find the sum of weights of cats.

```
SELECT SUM(weight)
FROM pet
WHERE species = "Cat";
```

```
+-----+
| SUM(weight) |
+-----+
|          23.1 |
+-----+
1 row in set (0.46 sec)
```

- COUNT : This function will count the number of row which satisfy given condition.

Example

- a. Display total number of pets having weight less than 20kg.

```
SELECT COUNT(*)
FROM pet
WHERE weight<20;
```

```
+-----+
| COUNT(*) |
+-----+
|          2 |
+-----+
1 row in set (0.03 sec)
```

b. Display the number of species of pets.

```
SELECT COUNT(DISTINCT species)
FROM pet;
```

This will count only non repeating values of the column.

```
+-----+
| COUNT(DISTINCT species) |
+-----+
|                2 |
+-----+
1 row in set (0.12 sec)
```

1.3 Single Row Functions

Single row functions operate on a single value to return a single value. They can accept one or more arguments but return only one result per row.

The single row functions are categorized in three categories

- Math Functions
- Text Functions
- Date Functions

We will be using following 'Single' table to demonstrate the functions.

```
+-----+-----+-----+-----+-----+-----+
| sno | name      | dob      | subject | height | weight |
+-----+-----+-----+-----+-----+-----+
| 12 | Rajesh Nikam | 1992-02-03 | 3 | 165.00 | 67.40 |
| 13 | Praveen Raja | 1991-11-03 | 3 | 159.00 | 71.40 |
| 16 | Anil Nair    | 1987-10-23 | 4 | 170.00 | 81.40 |
| 18 | Prachi Karve | 1988-04-21 | 6 | 160.00 | 65.20 |
| 19 | Medha Mishra | 1998-04-10 | 4 | 140.00 | 60.70 |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)
```

1.3.1 Math Functions

- Power

This function will return the x^y . The syntax of the function is

power(x,y)

```
mysql> select power(2,4);
+-----+
| power(2,4) |
+-----+
|          16 |
+-----+
1 row in set (0.04 sec)
```

The function also can be used with the table

```
mysql> select power(2,subject) from single;
+-----+
| power(2,subject) |
+-----+
|                8 |
|                8 |
|               16 |
|               64 |
|               16 |
+-----+
5 rows in set (0.00 sec)
```

- Round

This function will round off the value. The standard syntax is

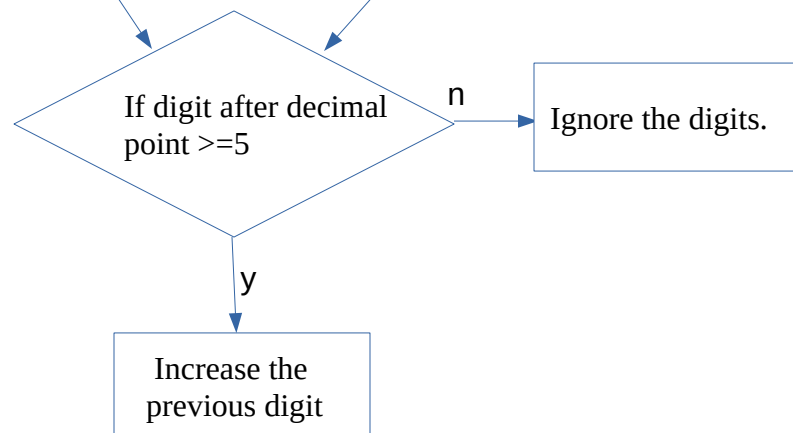
round (x) : Rounds off to integer.

round(x,d) : Rounds off the fraction with d number of digits.

```
mysql> select round(34.32);
+-----+
| round(34.32) |
+-----+
|           34 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> select round(34.67);
+-----+
| round(34.67) |
+-----+
|           35 |
+-----+
1 row in set (0.00 sec)
```

Number of
digits not
specified
therefore
taken as 0.



```
mysql> select round(35.5234,1);
+-----+
| round(35.5234,1) |
+-----+
|          35.5 |
+-----+
1 row in set (0.04 sec)
```

D=1 indicates that 1 digit should be left in the result. As the second digit from decimal place is <5 it is ignored.

```
mysql> select round(35.582,1);
+-----+
| round(35.582,1) |
+-----+
|          35.6 |
+-----+
1 row in set (0.00 sec)
```

D=1 indicates that 1 digit should be left in the result. As the second digit from decimal place is >=5 the next digit is incremented.

```
mysql> select round(35.582,-1);
+-----+
| round(35.582,-1) |
+-----+
|          40 |
+-----+
1 row in set (0.00 sec)
```

Negative number works with the digit before decimal place. As it is >=5 previous digit is increased by 1. Therefore result is 40.

```
mysql> select round(334.54,-2);
+-----+
| round(334.54,-2) |
+-----+
|          300 |
+-----+
1 row in set (0.00 sec)
```

One more example with negative d value. Now digit at 10s place is checked and as it is <5 it is ignored.

```
mysql> select round(weight) from single;
+-----+
| round(weight) |
+-----+
|          67 |
|          71 |
|          81 |
|          65 |
|          61 |
+-----+
5 rows in set (0.00 sec)
```

Round function on the column of the table.

- Mod

This function is used to find remainder of a division. The syntax of the function is

`mod(x , y)`

The result will be the remainder of x / y .

```
mysql> select mod(5,2);
+-----+
| mod(5,2) |
+-----+
|          1 |
+-----+
1 row in set (0.00 sec)
```

Remainder
of 5/2 is 1,
therefore
result is 1.

The function can be used with table column as shown in the example.

```
mysql> select mod(subject,3) from single;
+-----+
| mod(subject,3) |
+-----+
|                0 |
|                0 |
|                1 |
|                0 |
|                1 |
+-----+
5 rows in set (0.00 sec)
```

1.3.2 Text functions

The text functions work on strings in the table like name etc.

- Ucase/Upper

Both these functions convert the string to upper case that is capital letters.

```
mysql> select upper("Convert to Upper Case");
+-----+
| upper("Convert to Upper Case") |
+-----+
| CONVERT TO UPPER CASE          |
+-----+
1 row in set (0.03 sec)
```

While working with table the actual values do not change only the names are displayed in uppercase.

```
mysql> select upper(name) from single;
+-----+
| upper(name) |
+-----+
| RAJESH NIKAM |
| PRAVEEN RAJA |
| ANIL NAIR    |
| PRACHI KARVE |
| MEDHA MISHRA |
+-----+
5 rows in set (0.00 sec)
```


- Lcase / Lower

Both these functions convert the string to lower case that is small letters.

```
mysql> select lower("Convert to Lower Case");
+-----+
| lower("Convert to Lower Case") |
+-----+
| convert to lower case          |
+-----+
1 row in set (0.00 sec)
```

Either lcase or lower can be used with same effect.

```
mysql> select lcase(name) from single;
+-----+
| lcase(name) |
+-----+
| rajesh nikam |
| praveen raja |
| anil nair    |
| prachi karve |
| medha mishra |
+-----+
5 rows in set (0.00 sec)
```

- Mid / Substring / Substr

These function display the specified number of characters from the middle of the string. The syntax of the function is

Substr (str, m, n)

There are 3 arguments. The first argument is the source string. The second argument is the position of the first character to be displayed and the last argument is number of characters to be displayed.

```
mysql> select mid("Hello World",1,3);
+-----+
| mid("Hello World",1,3) |
+-----+
| Hel                    |
+-----+
1 row in set (0.07 sec)
```

Starting point is 1 so first character and 3 characters from there.

```
mysql> select mid("Hello World",3);
+-----+
| mid("Hello World",3) |
+-----+
| llo World            |
+-----+
1 row in set (0.01 sec)
```

When number of characters are not given then complete string from 3rd character is displayed.

```
mysql> select mid("Hello World",-3);
+-----+
| mid("Hello World",-3) |
+-----+
| rld                |
+-----+
1 row in set (0.05 sec)
```

Negative value is taken from end of string. So -3 displays last three characters.

```
mysql> select mid("Hello World",-7,3);
+-----+
| mid("Hello World",-7,3) |
+-----+
| o W                      |
+-----+
1 row in set (0.00 sec)
```

From -7 three characters

The diagram shows the counting positions both positive and negative.

H	e	l	l	o		W	o	r	l	d
0	1	2	3	4	5	6	7	8	9	10
-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```
mysql> select substr(name,2,3) from single;
+-----+
| substr(name,2,3) |
+-----+
| aje              |
| rav              |
| nil              |
| rac              |
| edh              |
+-----+
5 rows in set (0.59 sec)
```

From second character, three characters are displayed.

- Length

This function displays the number of characters in a string.

The syntax of the function is

Length(str)

```
mysql> select length("Hello World");
+-----+
| length("Hello World") |
+-----+
| 11                     |
+-----+
1 row in set (0.06 sec)
```

The number of characters of the string i.e. 11 displayed.

```
mysql> select length(name) from single;
+-----+
| length(name) |
+-----+
|          12  |
|          12  |
|           9  |
|          12  |
|          12  |
+-----+
5 rows in set (0.00 sec)
```

Number of
characters
in each
name.

- Left

This function displays the specified number of characters (n) from the left side of string.

The syntax for the function is

left (str, n)

n is the number of characters to display.

```
mysql> select left("Hello World",3);
+-----+
| left("Hello World",3) |
+-----+
| Hel                    |
+-----+
1 row in set (0.03 sec)
```

Three characters
from left selected.

```
mysql> select left(name,2)from single;
+-----+
| left(name,2) |
+-----+
| Ra           |
| Pr           |
| An           |
| Pr           |
| Me           |
+-----+
5 rows in set (0.00 sec)
```

Two characters from left
selected.

- Right

This function displays the specified number of characters (n) from the right side of string.

The syntax for the function is

`right (str, n)`

n is the number of characters to display.

```
mysql> select right("Hello World",3);
+-----+
| right("Hello World",3) |
+-----+
| rld                    |
+-----+
1 row in set (0.00 sec)
```

Three characters from right selected.

```
mysql> select right(name,2)from single;
+-----+
| right(name,2) |
+-----+
| am            |
| ja            |
| ir            |
| ve            |
| ra            |
+-----+
5 rows in set (0.00 sec)
```

Two characters from right selected.

- Instr

This function displays the position of the first occurrence of substring substr in string str.

The syntax of this function is

`instr (str, substr)`

```
mysql> select instr("Hello World","l");
+-----+
| instr("Hello World","l") |
+-----+
| 3                        |
+-----+
1 row in set (0.01 sec)
```

The first occurrence of "l" is at third character.

```
mysql> select instr(name,"a")from single;
+-----+
| instr(name,"a") |
+-----+
|                2 |
|                3 |
|                1 |
|                3 |
|                5 |
+-----+
5 rows in set (0.00 sec)
```

Position of first occurrence of "a".

```
mysql> select instr("Hello World","a");
+-----+
| instr("Hello World","a") |
+-----+
|                        0 |
+-----+
1 row in set (0.00 sec)
```

When the substring is not found it displays 0.

- Ltrim

This function removes the leading spaces from left side of the the string.

The syntax of the function is

`ltrim (str)`

```
mysql> select "      Hello World";
+-----+
| Hello World |
+-----+
1 row in set (0.00 sec)

mysql> select ltrim("      Hello World");
+-----+
| ltrim("      Hello World") |
+-----+
| Hello World |
+-----+
1 row in set (0.00 sec)
```

Original string with spaces on left side.

Leading space removed using ltrim function.

- Rtrim

This function removes the trailing spaces from right side of the the string.

The syntax of the function is

`rtrim (str)`

```
mysql> select rtrim("      Hello World      ") as result;
+-----+
| result |
+-----+
|      Hello World |
+-----+
1 row in set (0.00 sec)
```

Trailing spaces removed from the string.

```
mysql> select rtrim(name) as result from single;
+-----+
| result |
+-----+
| Rajesh Nikam |
| Praveen Raja |
| Anil Nair |
| Prachi Karve |
| Medha Mishra |
+-----+
5 rows in set (0.00 sec)
```

As there are no trailing spaces so the strings are displayed as they are.

- Trim

This function removes the leading and trailing spaces from the string.

The syntax of the function is

`trim (str)`


```
mysql> select trim("      Hello World      ") as result;
+-----+
| result |
+-----+
| Hello World |
+-----+
1 row in set (0.00 sec)
```

Observe that the space in between the two words is not removed. Only leading and trailing spaces are removed.

WORKING WITH MULTIPLE TABLES

1. Creating connected multiple tables

Till now we have been working on a single table at a time. But keeping data in a single table can make it bulky and also management of data difficult. Therefore often very big data is divided into multiple connected table.

1.1 Foreign Key

When we have multiple table and we need to establish connection we need to use foreign key. A FOREIGN KEY is a key used to link two tables together.

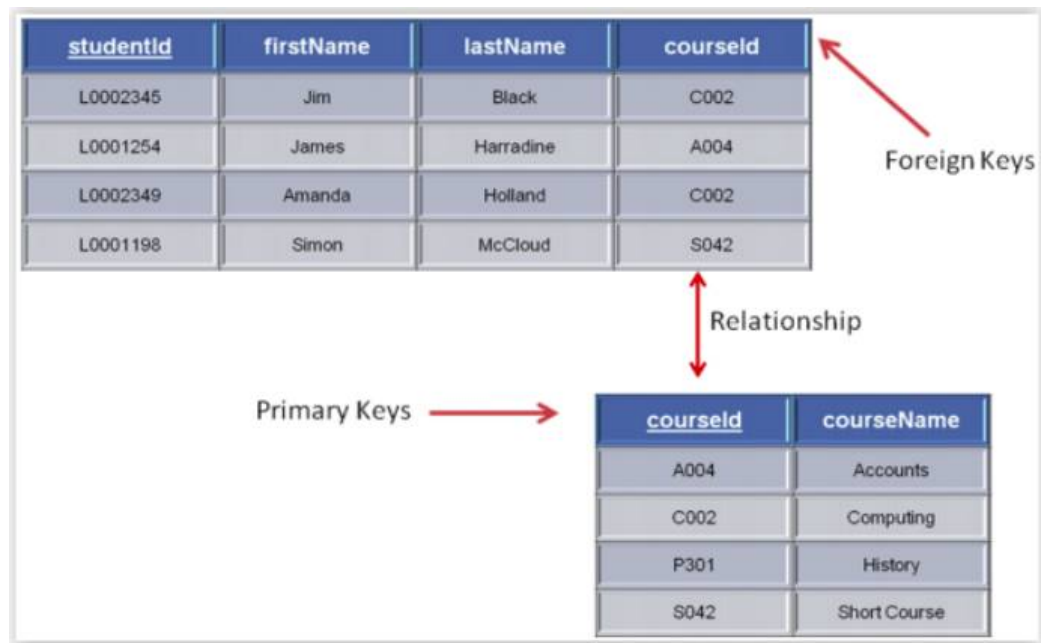
Let us recall the definition of foreign key, A FOREIGN KEY is a field (or collection of fields) in one table that refers to the PRIMARY KEY in another table.

The table containing the foreign key is called the child table, and the table containing the candidate key is called the referenced or parent table.

A foreign key is defined in a table using Referential constraint.

1.2 Referential Constraint

Referential integrity requires that a foreign key must have a matching primary key. This constraint is specified between two tables (parent and child); it maintains the correspondence between rows in these tables. It means the reference from a row in one table to another table must be valid.



This is the example that we are going to discuss and create tables.

Here the parent table is COURSE and the child or dependent table is STUDENT.

We will create the parent table first.

```
mysql> CREATE TABLE course(
  -> courseid char(5) primary key,
  -> coursename varchar(20));
Query OK, 0 rows affected (0.75 sec)

mysql> desc course;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| courseid   | char(5)       | NO   | PRI | NULL    |       |
| coursename | varchar(20)   | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Now we need to create the child table with the Foreign Key constant. The syntax for this is

```
FOREIGN KEY [foreign_key_name] (column_name, ...)
REFERENCES parent_table(column_name,...)
```

For student table the command to create table would be

```
CREATE TABLE student (
```



```

studentid char(8) primary key,
fname varchar(20),
lname varchar(20),
courseid varchar(20),
CONSTRAINT FK_course FOREIGN KEY (courseid)
REFERENCES course(courseid)
);

```

```

mysql> CREATE TABLE student (
-> studentid char(8) primary key,
-> fname varchar(20),
-> lname varchar(20),
-> courseid varchar(20),
-> CONSTRAINT FK_course FOREIGN KEY (courseid)
-> REFERENCES course(courseid)
-> );
Query OK, 0 rows affected (0.48 sec)

mysql> desc student;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| studentid | char(8) | NO | PRI | NULL | |
| fname | varchar(20) | YES | | NULL | |
| lname | varchar(20) | YES | | NULL | |
| courseid | varchar(20) | YES | MUL | NULL | |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

```

Observe the foreign key of the table.

Now let us add data in these table before we start working with queries.

```

mysql> select * from course;
+-----+-----+
| courseid | coursename |
+-----+-----+
| A004 | Computer Science |
| A005 | IP |
+-----+-----+
2 rows in set (0.01 sec)

mysql> select * from student;
+-----+-----+-----+-----+
| studentid | fname | lname | courseid |
+-----+-----+-----+-----+
| S0002341 | Raman | Kumar | A005 |
| S0003413 | Varun | Kishore | A004 |
| S0003621 | Nisha | Pandey | A005 |
+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

These are our tables after inserting data.

There are two main conditions that the foreign key constraint enforces.

- If a row from the parent table has a matching row in the child table, MySQL rejects deleting or updating rows in the parent table. Let us try that, by deleting a row from course table.

```
mysql> delete from course where courseid="A004";
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key
constraint fails (`classroom`.`student`, CONSTRAINT `FK_course` FOREIGN
KEY (`courseid`) REFERENCES `course` (`courseid`))
```

- No row in child table will have Foreign Key value that is not present in the parent table. So if we try to insert a row with Foreign Key

```
mysql> INSERT INTO student VALUES("S0004323","Paresh","Patel","A006");
ERROR 1452 (23000): Cannot add or update a child row: a foreign key
constraint fails (`classroom`.`student`, CONSTRAINT `FK_course` FOREIGN
KEY (`courseid`) REFERENCES `course` (`courseid`))
```

1.3 Cartesian Products

Cartesian product (also called Cross Join) of two tables is a table obtained by pairing up each row of one table with each row of the other table.

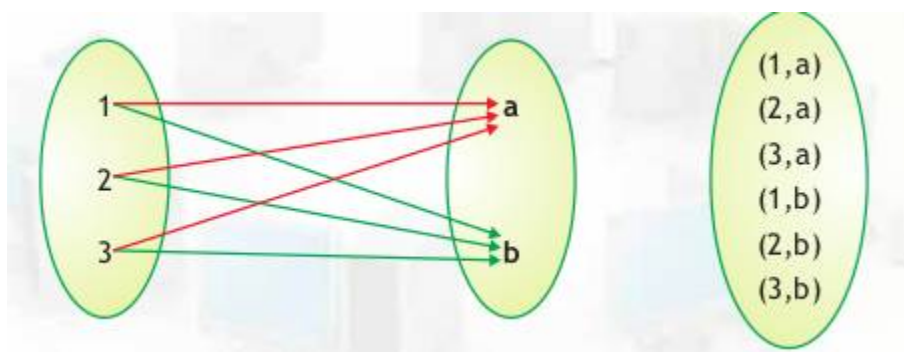
Observe the command given below

```
mysql> select * from student, course;
```

studentid	fname	lname	courseid	courseid	coursename
S0002341	Raman	Kumar	A005	A004	Computer Science
S0002341	Raman	Kumar	A005	A005	IP
S0003413	Varun	Kishore	A004	A004	Computer Science
S0003413	Varun	Kishore	A004	A005	IP
S0003621	Nisha	Pandey	A005	A004	Computer Science
S0003621	Nisha	Pandey	A005	A005	IP

6 rows in set (0.00 sec)

This can be viewed pictorially as



Every row of parent table is paired with each other row in child table. Notice that the arrows indicate the 'ordered pairing'.

The number of columns in the Cartesian product is the sum of the number of columns in both the tables. The number of rows in the Cartesian product is multiplication of number of rows in both the tables.

If the order of the tables in the FROM clause is changed the result remains same. Only the order of columns change.

```
mysql> select * from course,student;
+-----+-----+-----+-----+-----+-----+
| courseid | coursenam | studentid | fname | lname | courseid |
+-----+-----+-----+-----+-----+-----+
| A004     | Computer  | S0002341  | Raman | Kumar | A005     |
| A005     | IP        | S0002341  | Raman | Kumar | A005     |
| A004     | Computer  | S0003413  | Varun | Kishore | A004     |
| A005     | IP        | S0003413  | Varun | Kishore | A004     |
| A004     | Computer  | S0003621  | Nisha | Pandey | A005     |
| A005     | IP        | S0003621  | Nisha | Pandey | A005     |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)
```

The complete Cartesian product of two or more tables is, generally, not used directly. Sometimes the complete Cartesian product of two tables may give some confusing information also. If you observe the result of the above queries, even though Raman Kumar has course A005 but there is a row that pairs him with A004 also.

But we can extract meaningful information from the Cartesian product by placing some conditions in the statement. These are called joins.

1.4 Joins

Joins help retrieving data from two or more database tables. The tables are mutually related using primary and foreign keys.

1.4.1 Equi-Join

SQL EQUI JOIN performs a JOIN against equality or matching column(s) values of the associated tables. An equal sign (=) is used as comparison operator in the where clause to refer equality.

General syntax of equi-join is

```
SELECT column_list
FROM table1, table2....
WHERE table1.column_name = table2.column_name;
```

These are
two tables
that

One of these columns in
condition is a primary key of a
table and other is a foreign key
of another table.

Example

Let us see the two tables we had created earlier again to work with equi-join.

```
mysql> select * from student;
+-----+-----+-----+-----+
| studentid | fname | lname | courseid |
+-----+-----+-----+-----+
| S0002312 | Parag | Gupta | A004      |
| S0002542 | Rupa  | Gore  | A005      |
| S0003536 | Meera | Mishra | A004      |
| S0007732 | Sheela | M      | A006      |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

```
mysql> select * from course;
+-----+-----+
| courseid | coursename |
+-----+-----+
| A003     | AI         |
| A004     | Computer Science |
| A005     | IP         |
| A006     | IT         |
+-----+-----+
4 rows in set (0.00 sec)
```

Display the name of the course taken by Parag.

```
SELECT coursename
FROM course,student
WHERE course.courseid = student.courseid and fname="Parag";
```

Course.courseid is
primary key and
student.courseid is
foreign key.

```
mysql> select coursename from course,student where course.courseid = student.courseid and fname = "parag";
+-----+
| coursename |
+-----+
| Computer Science |
+-----+
1 row in set (0.28 sec)
```

Columns which are common in the two tables are accessed as tablename.columnname to avoid ambiguity.

This is additional condition on Equi-join.

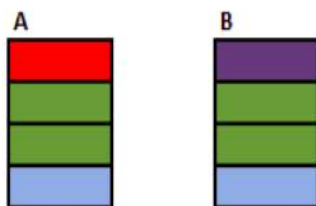
We have seen in previous lesson that a column can be given a meaningful name called column alias. Similarly a table also can be give a table alias. This is normally done to shorten the names of table while writing query. The following query demonstrates the usage of table alias.

```
mysql> select coursename from course c,student s where c.courseid = s.courseid and fname = "parag";
+-----+
| coursename |
+-----+
| Computer Science |
+-----+
1 row in set (0.62 sec)
```

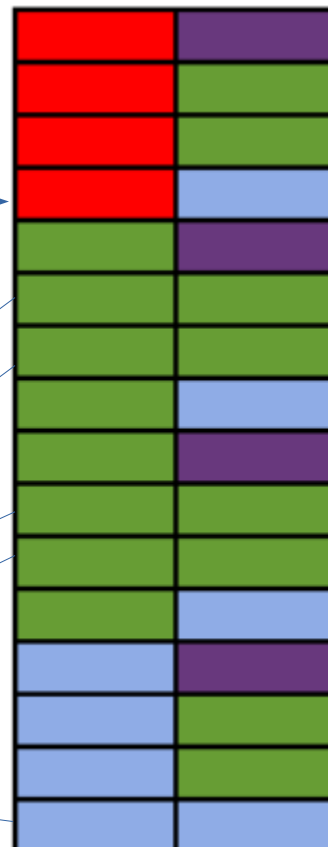
Course has alias name c and student has alias name s. The same alias name is used further in the query.

Pictorial representation is

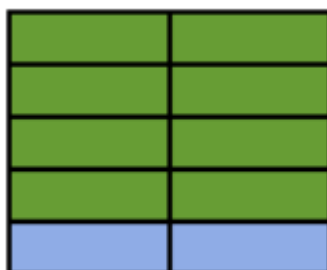
Source Tables



Cartesian Product



Equi - Join



The table view for the same is

mysql> select * from student;				mysql> select * from course;	
studentid	fname	lname	courseid	courseid	coursename
S0002312	Parag	Gupta	A004	A003	AI
S0002542	Rupa	Gore	A005	A004	Computer Science
S0003536	Meera	Mishra	A004	A005	IP
S0007732	Sheela	M	A006	A006	IT
4 rows in set (0.00 sec)				4 rows in set (0.00 sec)	

Equi join can be used for UPDATE queries also.

Example

Change the course of Rupa to AI.

```
UPDATE course,student
SET coursename = "AI"
WHERE course.courseid = student.courseid and fname="Rupa";
```

```
mysql> UPDATE course,student SET coursename="AI" WHERE course.courseid = student
.courseid and fname="Rupa";
Query OK, 1 row affected (0.74 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

The table after the updating is

mysql> select * from student;			
studentid	fname	lname	courseid
S0002312	Parag	Gupta	A004
S0002542	Rupa	Gore	A005
S0003536	Meera	Mishra	A004
S0007732	Sheela	M	A006
4 rows in set (0.00 sec)			

Observe
the change.

1.4.2 Natural Join

A natural join is a type of equi join which occurs implicitly by comparing all the same names columns in both tables. The join result has only one column for each pair of equally named columns.

```
mysql> select * from cou natural join student;
+-----+-----+-----+-----+-----+
| courseid | coursename | studentid | fname | lname |
+-----+-----+-----+-----+-----+
| A004 | Computer Science | S0002312 | Parag | Gupta |
| A005 | IP | S0002542 | Rupa | Gore |
| A004 | Computer Science | S0003536 | Meera | Mishra |
| A006 | IT | S0007732 | Sheela | M |
+-----+-----+-----+-----+-----+
4 rows in set (0.03 sec)

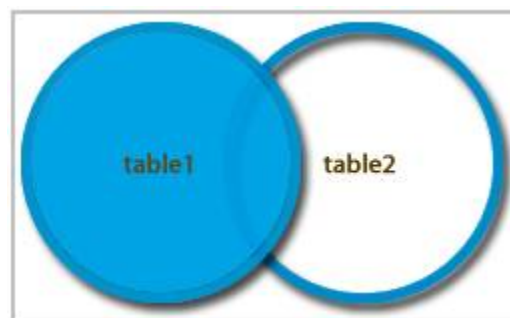
mysql> select * from cou,student where cou.courseid=student.courseid;
+-----+-----+-----+-----+-----+-----+
| courseid | coursename | studentid | fname | lname | courseid |
+-----+-----+-----+-----+-----+-----+
| A004 | Computer Science | S0002312 | Parag | Gupta | A004 |
| A005 | IP | S0002542 | Rupa | Gore | A005 |
| A004 | Computer Science | S0003536 | Meera | Mishra | A004 |
| A006 | IT | S0007732 | Sheela | M | A006 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

1.4.3 Left Join

The SQL LEFT JOIN (specified with the keywords LEFT JOIN and ON) joins two tables and fetches all matching rows of two tables for which the SQL-expression is true, plus rows from the first table that do not match any row in the second table.

```
SELECT *
FROM table1
LEFT [ OUTER ] JOIN table2
ON table1.column_name=table2.column_name;
```

Pictorially Left join can be shown as

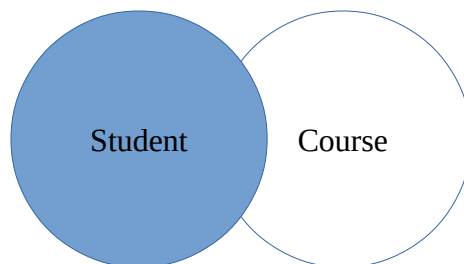


If the two tables are

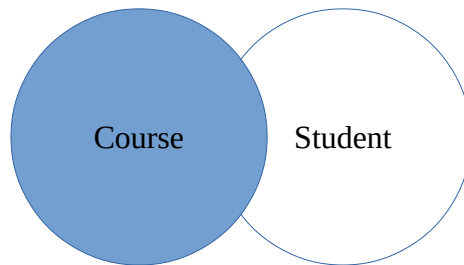

```
mysql> select * from course;
+-----+-----+
| courseid | coursename |
+-----+-----+
| A003     | AI         |
| A004     | Computer Science |
| A005     | AI         |
| A006     | IT         |
| A007     | SCRATCH    |
+-----+-----+
5 rows in set (0.00 sec)

mysql> select * from student;
+-----+-----+-----+-----+
| studentid | fname | lname | courseid |
+-----+-----+-----+-----+
| S0002312  | Parag | Gupta | A004     |
| S0002542  | Rupa  | Gore  | A005     |
| S0003536  | Meera | Mishra | A004     |
| S0007732  | Sheela | M      | A006     |
+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

A left join takes all the values from the first table, and looks for matches in the second table. If it finds a match, it adds the data from the second table; if not, it adds missing values as NULL.



```
mysql> select * from student left join course on student.courseid = course.courseid;
+-----+-----+-----+-----+-----+-----+
| studentid | fname | lname | courseid | courseid | coursename |
+-----+-----+-----+-----+-----+-----+
| S0002312  | Parag | Gupta | A004     | A004     | Computer Science |
| S0003536  | Meera | Mishra | A004     | A004     | Computer Science |
| S0002542  | Rupa  | Gore  | A005     | A005     | AI               |
| S0007732  | Sheela | M      | A006     | A006     | IT               |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.04 sec)
```



```
mysql> select * from course left join student on student.courseid = course.courseid;
```

courseid	coursename	studentid	fname	lname	courseid
A004	Computer Science	S0002312	Parag	Gupta	A004
A005	AI	S0002542	Rupa	Gore	A005
A004	Computer Science	S0003536	Meera	Mishra	A004
A006	IT	S0007732	Sheela	M	A006
A003	AI	NULL	NULL	NULL	NULL
A007	SCRATCH	NULL	NULL	NULL	NULL

```
6 rows in set (0.00 sec)
```

Example

Display number of course which are not taken by any student.

```
SELECT count(*)
FROM course
LEFT JOIN student ON student.courseid = course.courseid
WHERE studentid is null;
```

```
mysql> select count(*) from course left join student on student.courseid = course.courseid where studentid is null;
```

count(*)
2

```
1 row in set (0.00 sec)
```

1.5 Union

Union is an operation of combining the output of two SELECT statements. Union of two SELECT statements can be performed only if their outputs contain same number of columns and data types of corresponding columns are also the same.

The syntax of UNION in its simplest form is:

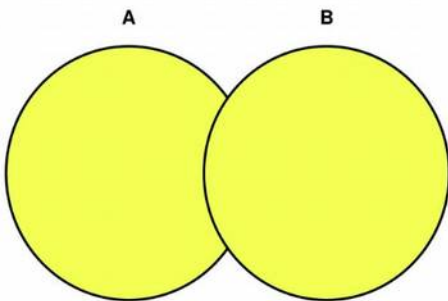
```
SELECT <select_list>
```

```

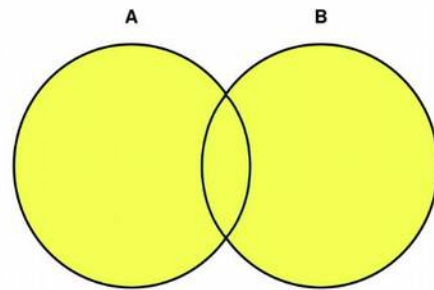
FROM <tablename>
[WHERE <condition> ]
UNION [ALL]
SELECT <select_list>
FROM <tablename>
[WHERE <condition> ];

```

Union does not display any duplicate rows unless ALL is specified with it.



The UNION operator returns results from both queries after eliminating duplications.



The UNION ALL operator returns results from both queries, including all duplications.

Example:

If a course id exists with same course code in both the tables, its code will be displayed only once - because Union does not display duplicate rows. If we explicitly want the duplicate rows, then we can enter the statement with UNION ALL:

```

mysql> select courseid from student union select courseid from course;
+-----+
| courseid |
+-----+
| A004     |
| A005     |
+-----+
2 rows in set (0.00 sec)

mysql> select courseid from student union all select courseid from course;
+-----+
| courseid |
+-----+
| A004     |
| A005     |
| A005     |
| A004     |
| A005     |
+-----+
5 rows in set (0.00 sec)

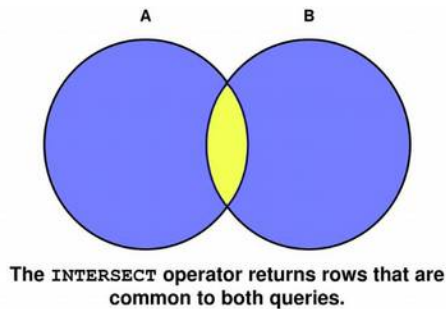
```

The UNION operator is used to combine the result-set of two or more SELECT statements.

- Each SELECT statement within UNION must have the same number of columns
- The columns must also have similar data types
- The columns in each SELECT statement must also be in the same order

1.6 Intersection

Intersect operation is used to combine two SELECT statements, but it only returns the records which are common from both SELECT statements.



To use the INTERSECT operator for two queries, you follow these rules:

- The order and the number of columns in the select list of the queries must be the same.
- The data types of the corresponding columns must be compatible.

There is no direct intersection command in MySQL. Therefore we use a nested query as show in the example.

```
mysql> select courseid from student where courseid IN (select courseid from course);
```

courseid
A004
A005
A005

```
3 rows in set (0.03 sec)
```

```
mysql> select DISTINCT courseid from student where courseid IN (select courseid from course);
```

courseid
A004
A005

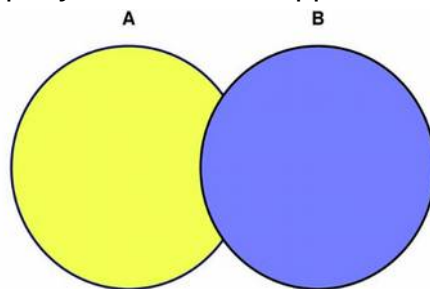
```
2 rows in set (0.02 sec)
```

How it works.

- The subquery returns the first result set.
- The outer query uses the IN operator to select only values that exist in the first result set. The DISTINCT operator ensures that only distinct values are selected.

1.7 Minus

The MINUS compares the results of two queries and returns distinct rows from the result set of the first query that does not appear in the result set of the second query.



The MINUS operator returns rows in the first query that are not present in the second query.

The syntax of the query is

```
SELECT select_list FROM table1
LEFT JOIN table2
ON join_predicate
WHERE table2.column_name IS NULL;
```

Example

```
mysql> select course.courseid from course left join student on
-> student.courseid = course.courseid where student.courseid is null;
+-----+
| courseid |
+-----+
| A003     |
| A007     |
+-----+
2 rows in set (0.00 sec)
```

There is two rows in course with courseid that is not present in student table. That means there are two courses that are not taken by any student.